



Universidade de Ciências e Tecnologia Joaquim Alberto Chissano

Curso: Engenharia Informática

Cadeira: Sistemas Operativos- II

Tema: CONCORRÊNCIA EM SISTEMAS OPERATIVOS

Nome dos Discentes:

Avelino Francisco Mucare Junior

Marlon Baptista Muchanga

Shelzia Jessiana Lima Matsinhe

Nome do Docente:

Msc. Belarmino Luís Botão

Maputo, Abril 2026

Índice

1.	Introdução	3
2.	Objectivos	4
2.1	Objectivo Geral	4
2.2	Objectivos Específicos	4
3.	Metodologia	5
4.	Desenvolvimento	6
4.1	Conceituação de Concorrência e Distinção Conceptual	6
4.2	Tipos de Concorrência: Processos e Threads.....	6
4.3	Funções Vitais da Concorrência na Gestão de Recursos	7
4.4	Importância Crítica da Concorrência nos Sistemas Modernos.....	8
4.5	Vantagens e Desafios da Implementação	8
4.5.1	Vantagens	8
4.5.2	Desvantagens e Custos (Overhead)	9
4.5.3	Problemas de Sincronização.....	9
5.	Conclusão	11
6.	Referências Bibliográficas.....	12

1. Introdução

A concorrência constitui um dos pilares centrais na arquitectura e funcionamento dos Sistemas Operativos (SO) contemporâneos. No contexto da computação, a concorrência refere-se à capacidade de um sistema gerir e executar múltiplas tarefas ou processos durante um período de tempo, criando a percepção de que estas ocorrem simultaneamente. É fundamental, desde logo, distinguir com precisão o conceito de "concorrência" de "paralelismo".

Enquanto o paralelismo implica a execução literal de duas ou mais tarefas no mesmo instante — o que exige hardware com processadores de múltiplos núcleos ou múltiplos processadores físicos —, a concorrência foca-se na gestão estrutural de múltiplas tarefas que progridem de forma sobreposta, podendo ser intercaladas (alternadas) pelo processador através de técnicas de escalonamento.

Com a evolução tecnológica e a popularização dos processadores multi-core, a concorrência deixou de ser apenas um recurso de optimização para tornar-se uma necessidade absoluta. Ela permite que os recursos do computador, como a Unidade Central de Processamento (CPU) e a memória, sejam mantidos ocupados de forma eficiente, evitando a ociosidade e garantindo uma melhor experiência ao utilizador. Segundo Silberschatz, Galvin e Gagne (2018), “a concorrência permite que vários processos progridam simultaneamente, aumentando a eficiência do sistema e a capacidade de resposta ao utilizador”.

O presente trabalho tem como objectivo principal compreender em profundidade o conceito de concorrência em Sistemas Operativos, analisando a sua operacionalidade, importância, bem como as vantagens e os desafios inerentes à sua implementação.

2. Objectivos

2.1 Objectivo Geral

Compreender de forma aprofundada o conceito de concorrência em Sistemas Operativos, analisando a sua operacionalidade e distinção em relação ao paralelismo, bem como avaliar a sua importância crítica na computação moderna, as vantagens na gestão de recursos e os desafios inerentes à sua implementação e sincronização.

2.2 Objectivos Específicos

- Definir claramente o conceito de concorrência, distinguindo-a de outros conceitos afins, nomeadamente o paralelismo;
- Identificar e caracterizar os principais tipos de concorrência ao nível de processos e threads (linhas de execução);
- Explicar as funções vitais da concorrência na optimização da gestão de recursos de hardware;
- Analisar a importância crítica da concorrência nos sistemas de computação modernos, incluindo servidores e sistemas distribuídos;
- Avaliar as vantagens e desvantagens da sua implementação, com um foco detalhado nos problemas de sincronização, tais como condições de corrida, impasses e inanição.

3. Metodologia

Para a elaboração deste trabalho, adoptou-se uma abordagem qualitativa e descritiva, suportada pelas seguintes metodologias:

1. **Pesquisa Bibliográfica:** Consulta e revisão de literatura especializada, incluindo livros de referência na área de Sistemas Operativos e artigos científicos recentes.
2. **Análise Descritiva:** Explicação detalhada dos conceitos teóricos apoiada por exemplos práticos do quotidiano computacional para facilitar a compreensão.
3. **Síntese Comparativa:** Comparação entre os diferentes mecanismos de concorrência e os desafios associados.

O trabalho baseia-se nas perspectivas de autores renomados como Andrew S. Tanenbaum, Abraham Silberschatz e William Stallings, garantindo um embasamento teórico sólido.

4. Desenvolvimento

4.1 Conceituação de Concorrência e Distinção Conceptual

A concorrência, em Sistemas Operativos, é a capacidade de o sistema lidar com várias actividades ao mesmo tempo. Estas actividades podem ser programas individuais (processos) ou segmentos de execução dentro de um mesmo programa (threads).

O mecanismo subjacente que permite a concorrência em sistemas com um único processador chama-se **Multiprogramação** ou **Time-sharing** (partilha de tempo). O Sistema Operativo alterna rapidamente a CPU entre os diversos processos. Esta troca, denominada "troca de contexto" (context switching), é tão veloz que, para o utilizador humano, parece que todas as aplicações estão a funcionar em simultâneo.

Segundo Tanenbaum e Bos (2015), "a concorrência envolve muitas coisas diferentes: estruturas de dados multiprocessadas, programas de execução paralela, interacção com hardware assíncrono. É uma forma de estruturar sistemas onde múltiplas actividades ocorrem de forma sobreposta no tempo".

Exemplo prático: Imagine um utilizador a trabalhar num computador onde o processador está a efectuar cálculos complexos para uma folha de cálculo (Excel). Ao mesmo tempo, o sistema está a baixar actualizações em segundo plano e o utilizador está a ouvir música. Mesmo que a CPU execute apenas uma instrução de cada vez (num sistema single-core), a concorrência permite que estas tarefas intercalem os seus ciclos de processamento e espera, parecendo todas activas e responsivas.

4.2 Tipos de Concorrência: Processos e Threads

A concorrência pode ser classificada com base na unidade de execução, sendo essencial distinguir entre concorrência pesada (processos) e leve (threads).

1. Concorrência ao Nível de Processos (Process-Level Concurrency)

Ocorre quando o Sistema Operativo gere múltiplos programas independentes simultaneamente. Cada processo possui o seu próprio espaço de endereçamento de memória, o que significa que variáveis, código e dados são isolados. A comunicação entre processos é mais complexa e exige mecanismos específicos (IPC).

- **Exemplo:** Abrir o Microsoft Word, o Google Chrome e o VLC Media Player ao mesmo tempo. Se o Word sofrer uma falha ("crashar") e fechar inesperadamente, isso não afecta necessariamente o Chrome, pois estão em áreas de memória separadas e protegidas.

2. Concorrência ao Nível de Threads (Thread-Level Concurrency)

As threads (linhas de execução ou "tarefas leves") são subunidades de um processo. Múltiplas threads dentro do mesmo processo partilham a mesma área de memória e recursos do processo (como variáveis globais e ficheiros abertos), mas cada uma tem o seu próprio ponteiro de instrução, pilha de execução (stack) e registos.

Exemplo: Um processador de texto que verifica a ortografia ao mesmo tempo que permite ao utilizador digitar texto. A tarefa de "escrever" e a tarefa de "verificar ortografia" são duas threads concorrentes no mesmo processo. Aqui, o custo de criação e troca de contexto é menor, mas o risco de erros de partilha de dados é maior.

4.3 Funções Vitais da Concorrência na Gestão de Recursos

A concorrência não é apenas uma característica, mas uma função estratégica do Sistema Operativo com os seguintes papéis fundamentais:

- **Maximizar a Utilização da CPU:** A CPU é exponencialmente mais rápida que os dispositivos de entrada/saída (I/O), como discos rígidos, redes ou impressoras. A concorrência permite que, enquanto um processo espera por dados do disco (estado de bloqueio), a CPU seja imediatamente atribuída a outro processo que esteja pronto para calcular (estado pronto/pronto a executar).
- **Facilitar a Multitarefa:** Permite que os utilizadores executem múltiplas aplicações de forma produtiva, alternando entre elas sem a necessidade de encerrar a anterior.
- **Melhorar a Capacidade de Resposta:** Em sistemas interactivos, garante que o sistema permanece responsivo. Por exemplo, se uma aplicação está a processar um vídeo pesado, a concorrência permite que o rato e o teclado continuem a funcionar noutras janelas.

Exemplo prático: Quando um utilizador está a copiar um ficheiro grande de 10 GB para uma pen drive (operação lenta de I/O), a concorrência permite que o sistema utilize esse tempo de espera para actualizar

o relógio do sistema, receber mensagens de email ou manter a animação da barra de progresso fluida no ecrã.

4.4 Importância Crítica da Concorrência nos Sistemas Modernos

A concorrência é indispensável na era da computação moderna. A sua importância justifica-se pelos seguintes factores:

1. **Aproveitamento de Processadores Multi-core:** Os processadores modernos possuem 4, 8, 16 ou mais núcleos. Sem concorrência (e paralelismo), a maioria desses núcleos ficaria inactiva. A concorrência permite distribuir tarefas (threads) pelos diferentes núcleos, multiplicando a capacidade de processamento.
2. **Sistemas Distribuídos e Redes:** A internet funciona com base em servidores concorrentes. Um servidor web (como o Google ou serviços de banco online) atende a milhões de pedidos ao mesmo tempo. Se não fosse concorrente, cada pedido teria de esperar o anterior terminar, tornando a internet inutilizável para o escalonamento global.
3. **Aplicações em Tempo Real:** Sistemas críticos como controlo de tráfego aéreo, monitorização hospitalar ou sistemas de frenagem de automóveis requerem concorrência para processar dados de múltiplos sensores instantaneamente e agir dentro de prazos estritos (deadlines).

Segundo Silberschatz et al. (2018), “sem concorrência, os sistemas modernos seriam extremamente lentos e ineficientes, incapazes de atender às demandas de processamento simultâneo de dados”.

4.5 Vantagens e Desafios da Implementação

Apesar dos benefícios óbvios, a concorrência introduz complexidade significativa e problemas críticos que o programador e o Sistema Operativo devem resolver.

4.5.1 Vantagens

- **Desempenho:** Aumento significativo da velocidade de execução de tarefas e吞吐 (throughput) do sistema.
- **Eficiência:** Melhor aproveitamento dos recursos de hardware (CPU, memória), minimizando o tempo de inactividade (idle time).

- **Responsividade:** O sistema reage mais rapidamente aos comandos do utilizador, mesmo sob carga pesada.
- **Partilha de Recursos:** Múltiplos utilizadores ou aplicações podem partilhar ficheiros, periféricos (como impressoras) e dados de forma organizada e segura.

4.5.2 Desvantagens e Custos (Overhead)

- **Complexidade de Programação:** Escrever programas concorrentes é muito mais difícil do que programas sequenciais. O programador deve prever múltiplos cenários de execução intercalada.
- **Dificuldade de Depuração (Debugging):** Erros como race conditions são difíceis de reproduzir, pois dependem do tempo (são não-determinísticos). Um erro pode aparecer uma vez e nunca mais, tornando a sua correcção desafiadora.
- **Overhead (Custo Extra):** O Sistema Operativo gasta tempo e recursos para gerir a troca de contexto entre processos/threads (guardar o estado de um e carregar o do outro), o que consome ciclos de CPU.
- **Necessidade de Sincronização:** É obrigatório usar mecanismos como Semáforos, Mutexes ou Monitores para proteger dados partilhados, o que, se mal utilizado, pode adicionar lentidão ao sistema.

Problemas de Sincronização

A implementação incorreta da concorrência leva a problemas clássicos:

1. **Condição de Corrida (Race Condition):** Ocorre quando dois ou mais processos acedem a dados partilhados simultaneamente, e o resultado final depende da ordem específica em que as instruções são executadas. Se houver interferência não controlada, os dados podem ser corrompidos.

Exemplo: Imagine dois processos, A e B, a lerem o saldo de uma conta bancária (que é 100 MT). Ambos decidem adicionar 50 MT e gravar o novo saldo. Se não houver sincronização, ambos podem ler 100, somar 50, e gravar 150. O saldo final será 150 em vez dos 200 correctos.

2. **Deadlock (Impasse):** Ocorre quando um conjunto de processos fica bloqueado porque cada um deles está a esperar por um recurso que está detido por outro processo no conjunto. É como um "empate" onde ninguém cede.

Exemplo: O Processo 1 tem a Impressora e quer o Scanner. O Processo 2 tem o Scanner e quer a Impressora. Ambos esperam eternamente um pelo outro.

3. **Starvation (Inanição ou Fome):** Ocorre quando um processo está pronto para executar, mas o Sistema Operativo nunca lhe atribui a CPU porque outros processos com "prioridade mais alta" ou mais agressivos estão sempre a tomar a vez.

Exemplo: Um utilizador comum a tentar imprimir um documento num servidor ocupado, mas o director da empresa está a imprimir documentos gigantes a cada 5 segundos. O documento do utilizador pode nunca ser impresso (inanição).

5. Conclusão

A concorrência é um elemento vital e inerente aos Sistemas Operativos modernos. Ela permite que as máquinas actuais, sejam elas computadores pessoais, telemóveis ou supercomputadores, realizem múltiplas tarefas de forma eficiente e "simultânea", proporcionando uma experiência de utilizador fluida e produtiva.

Ao longo deste trabalho, verificámos que a concorrência não é isenta de desafios. Problemas como condições de corrida, deadlocks e inanição representam riscos reais para a estabilidade e integridade dos dados, exigindo um cuidado meticuloso na programação e no desenho de algoritmos de escalonamento. No entanto, através do uso de mecanismos de sincronização robustos e de uma arquitectura bem planeada, estes problemas podem ser geridos e mitigados.

Em suma, os benefícios da concorrência, como o aumento de desempenho e a maximização do uso de hardware — especialmente numa era de processadores multi-core — superam largamente as suas desvantagens e complexidade inerentes. Para qualquer estudante ou profissional de informática, dominar estes conceitos é crucial para o desenvolvimento de software robusto e para a gestão eficiente de infraestruturas tecnológicas.

6. Referências Bibliográficas

- DEITEL, Paul; DEITEL, Harvey. Sistemas Operativos. 3.^a Edição. São Paulo: Pearson Prentice Hall, 2005.
- SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. Operating System Concepts. 10.^a Edição. Hoboken, NJ: Wiley, 2018.
- STALLINGS, William. Operating Systems: Internals and Design Principles. 9.^a Edição. Pearson, 2018.
- TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 4.^a Edição. Boston: Pearson, 2015.